Rapport révisé — Classification des véhicules, modèles et finitions

Date : 21 octobre 2026. Dépôt : `AchyOUATT/autoparc-backend`, branche locale `main`, commit `e8264c4f6fd6bcf092236b175c2c869cd27e19e4`.

1. Conclusion pour décision

Les trous viennent de trois causes distinctes : des données manquantes, des mises à jour qui ne parviennent pas correctement au cache mobile, et des règles qui peuvent rattacher un véhicule ou une consommation à une mauvaise référence.

Le catalogue versionné contient **248 entrées modèle/génération pour 174 couples marque–nom commercial**. **181 entrées, soit 73,0 %, n'ont aucune finition**. Dix marques n'ont aucune finition dans le seeder. Cela confirme un manque important, mais ce taux ne mesure pas la proportion de véhicules réels mal classés : il faut le pondérer par le parc et les garages effectivement utilisés.

La priorité proposée est de fiabiliser la propagation et les rattachements, puis de compléter les données à partir des besoins réels. Une séparation physique complète entre modèle, génération, variante et finition peut être utile, mais les défauts les plus urgents peuvent être corrigés sans attendre cette refonte.

Aucun changement applicatif, aucune migration et aucune correction de données n'ont été appliqués. Les fichiers de ce dossier constituent uniquement le rapport et ses annexes d'analyse.

2. Méthode, preuves et limites

L'analyse porte sur les seeders réellement utilisés, les migrations, les modèles, les validations d'écriture, les réponses API, la synchronisation, les imports, le décodage VIN, les compatibilités et les tests existants.

Les comptages proviennent d'un script Python exécuté sur les tableaux littéraux des seeders. Il relève les lignes sources, distingue les commentaires des données, contrôle les clés et produit les annexes CSV/JSON. Il ne lance pas les seeders Laravel et ne lit aucune base distante.

Le rapport distingue :

- **Constat établi dans le code** : chemin de données ou règle effectivement présent dans cette révision.
- **Effet à mesurer en production** : combien de fiches ou de téléphones sont touchés, et quels écrans consomment les champs concernés.
- **Évolution proposée** : choix de conception à décider, sans le présenter comme un défaut déjà démontré.

PHP et les dépendances Composer ne sont pas installés dans l'instance actuelle. **La suite PHPUnit n'a pas été exécutée pour ce rapport.** Les scénarios décrits sont établis par lecture du code ; leurs réponses HTTP exactes restent à reproduire avant un correctif. Aucun accès à la production ni au dépôt Flutter n'a été utilisé. Les caractéristiques automobiles et les gammes mondiales n'ont pas été revérifiées auprès des constructeurs : aucune nouvelle finition n'est déclarée authentique sur la seule base de cet audit.

3. État chiffré du catalogue versionné

Indicateur	Résultat	Interprétation
Marques avec modèles	25	Périmètre couvert par le seeder de modèles
Couples marque–nom commercial	174	Comptage des noms, pas des générations
Entrées modèle/génération	248	Le niveau de détail varie selon les modèles
Finitions/versions	258	Certaines lignes sont des désignations techniques
Entrées ayant au moins une finition	67, soit 27,0 %	Ne prouve pas la gamme complète
Entrées sans finition	181, soit 73,0 %	Manque confirmé dans le référentiel versionné
Marques sans aucune finition	10	Voir tableau ci-dessous
Générations non renseignées	33	En partie volontaire : plusieurs générations sont regroupées
Périodes de production sans fin renseignée	142	Pas une preuve d’obsolescence
Transitions partageant une même année	70	Ancienne fin = nouveau début ; ambiguïté avec des années inclusives
Doublons de clé modèle ou finition dans les seeders	0	Ne préjuge pas des doublons en production
Finitions pointant vers un modèle absent du seeder	0	Les rattachements statiques sont résolus

Couverture complète par marque

Marque	Entrées modèle/génération	Avec finition	Sans finition	Nombre de finitions
Toyota	29	14	15	65
Mazda	19	0	19	0
Nissan	14	3	11	10
Honda	11	0	11	0
Hyundai	12	2	10	7
Renault	11	1	10	4
Citroën	10	0	10	0
Opel	10	0	10	0
Kia	10	1	9	4
Peugeot	9	1	8	4
Mercedes-Benz	9	1	8	4
Suzuki	8	0	8	0
Fiat	8	0	8	0
Volkswagen	8	1	7	4
SsangYong	7	0	7	0
Mitsubishi	8	2	6	7

Marque	Entrées modèle/génération	Avec finition	Sans finition	Nombre de finitions
Chevrolet	5	0	5	0
Subaru	5	0	5	0
Ford	5	1	4	4
Land Rover	5	1	4	3
Jeep	4	1	3	3
Daihatsu	2	0	2	0
Isuzu	2	1	1	4
BMW	14	14	0	67
Lexus	23	23	0	68
Total	**248**	**67**	**181**	**258**

BMW et Lexus concentrent 135 des 258 lignes de finition, soit 52,3 %. Cette concentration ne démontre ni leur exhaustivité ni l'exactitude de tous les attributs.

L'annexe `modeles-et-finitions.csv` contient les 248 entrées avec génération, période, nombre et noms des finitions, ainsi que la ligne du seeder. Elle permet de filtrer les manques sans les ressaisir.

4. Défauts de fonctionnement à traiter en premier

F1 — Les enrichissements de marques et modèles peuvent rester invisibles en synchronisation incrémentale

Constat établi. `VehicleModelsSeeder` et `BrandsSeeder` utilisent `DB::table(...)->upsert()` sans écrire `updated_at`. Contrairement à un enregistrement Eloquent, cette opération Query Builder ne gère pas automatiquement les timestamps. Les timestamps des tables sont nullable. La synchronisation incrémentale ne prend que les lignes dont `updated_at > since`.

Une nouvelle ligne créée par ces seeders peut donc avoir un timestamp nul ; une ligne corrigée peut garder son ancien timestamp. Dans les deux cas, un téléphone ayant déjà synchronisé ne reçoit pas forcément l'enrichissement. Une synchronisation complète peut le renvoyer.

Conséquence : ajouter une référence dans le dépôt ne garantit pas qu'elle apparaisse dans une application déjà installée. Cela constitue une cause possible de trous perçus même lorsque la référence existe côté serveur.

Proposition : garantir une date de modification pour chaque changement réel, prévoir un rattrapage des anciennes lignes et vérifier la séquence complète : synchronisation initiale → ajout/correction → synchronisation incrémentale. Éviter de rendre tout le catalogue « modifié » à chaque démarrage sans changement de contenu.

Sources : `database/seeders/VehicleModelsSeeder.php:500`, `database/seeders/BrandsSeeder.php:60`, `app/Http/Controllers/Api/CatalogSyncController.php:82`, migration de référence `2026_01_01_000100_create_reference_tables.php:47`.

F2 — Une modification peut conserver une classification incompatible

Constat établi. Les contrôles marque–modèle et modèle–finition existent à la création. Dans le garage, ils vérifient surtout les couples envoyés ensemble, et non tous les liens de l'état final.

Exemples de chemins insuffisamment protégés : changer uniquement la marque conserve l'ancien modèle ; changer uniquement la finition ne vérifie pas son appartenance au modèle conservé ; changer uniquement le modèle peut laisser l'ancienne finition et l'ancienne motorisation. Le contrôleur applique directement les champs validés. Les clés étrangères indépendantes vérifient l'existence des références, pas leur cohérence mutuelle.

Le parc commercial conserve davantage de champs obligatoires à la mise à jour, mais une ancienne finition peut également subsister si le modèle est remplacé et que `trim_id` est omis.

Proposition : valider la fiche résultante avant écriture. Lorsqu'un changement invalide une dépendance, retourner une décision explicite à l'utilisateur ou une erreur de validation ; ne pas transférer silencieusement une finition ou une cote de consommation sur un autre modèle.

Sources : `app/Http/Requests/StoreOwnedVehicleRequest.php:103`, `UpdateOwnedVehicleRequest.php:38`, `app/Http/Controllers/Api/OwnedVehicleController.php:63`, `app/Http/Requests/StoreVehicleRequest.php:143`, `UpdateVehicleRequest.php:9`.

F3 — « Code moteur » désigne deux données incompatibles selon le chemin

Constat établi sur le contrat API. Le premier niveau `engine_code` du décodage VIN est le seul huitième caractère du VIN. Le champ NHTSA `EngineCode`, lorsqu'il existe, est exposé séparément dans `nhtsa.engine_code_nhtsa`. Le moteur de compatibilité compare pourtant un code complet comme `2AR-FE` ou `A25A-FKS` par égalité exacte.

Si le client recopie le champ supérieur dans la fiche du garage, il enregistre une donnée d'une autre nature et peut perdre les compatibilités spécifiques. **L'utilisation exacte de ce champ par Flutter reste à vérifier.** La disponibilité du champ NHTSA ne garantit pas non plus, à elle seule, une correspondance avec le référentiel de codes utilisé par les pièces.

Autre incohérence confirmée : la recherche publique normalise le code moteur en majuscules ; la saisie du garage ne le fait pas. Sur PostgreSQL, `2ar-fe` et `2AR-FE` peuvent donc produire des résultats différents.

Proposition : distinguer caractère du VIN et code moteur constructeur résolu, normaliser les codes aux points d'entrée, conserver source et confiance et mesurer les anciennes valeurs avant toute correction automatique.

Sources : `app/Services/VinDecodeService.php:145`, `:225`, `app/Services/CompatibilityService.php:92`, `app/Http/Requests/CompatiblePartsCriteriaRequest.php:67`, `StoreOwnedVehicleRequest.php:61`, `app/Models/OwnedVehicle.php:279`.

F4 — Une compatibilité peut être trop affirmative lorsque la variante est inconnue

Constat établi. Les filtres sur finition, carburant, motricité et code moteur ne sont appliqués que lorsque ces critères sont renseignés. Une recherche incomplète peut donc retourner plusieurs références propres à des variantes différentes. Ce comportement peut être utile pour une recherche large, mais ne suffit pas à confirmer une pièce pour un véhicule précis.

Le booléen `compatibility_ready` peut être vrai avec un modèle et le seul type « essence ». Cela ne permet pas toujours de distinguer deux moteurs essence d'une même génération. La réponse de compatibilité ne représente pas suffisamment la différence entre correspondance confirmée, candidat à vérifier et information insuffisante.

Proposition : qualifier la compatibilité selon les critères réellement nécessaires à chaque référence. Demander moteur ou motricité lorsqu'ils départagent les pièces, et conserver des résultats possibles sans les annoncer comme confirmés.

Les fitments `generated` sont explicitement synthétiques ; le service ne filtre pas leur source. Une purge existe : **leur présence actuelle en production n'est pas établie** et doit être mesurée avant conclusion.

Sources : `app/Services/CompatibilityService.php:71`, `app/Models/OwnedVehicle.php:297`, `app/Http/Controllers/Api/GarageCompatibilityController.php:36`, `app/Enums/FitmentSource.php:13`, `app/Support/FitmentPlanner.php:129`.

5. Limites de la classification et des données

D1 — Le niveau de détail des modèles n’est pas uniforme

Corolla est découpée en générations, tandis que Highlander, Sienna, Venza, Avalon ou Tacoma sont des entrées plus larges. Une partie des 33 générations nulles correspond volontairement à ce regroupement.

Les tests interdisent explicitement d’attribuer une seule génération à des entrées comme 208, Duster, Amarok, Sprinter et Alto qui en regroupent plusieurs. **Remplir les cases vides avec un code unique serait parfois une régression.** Une scission doit préserver les identifiants historiques ou prévoir des correspondances et une reclassification contrôlée.

Sources : `VehicleModelsSeeder.php:16`, `tests/Feature/GenerationsNommeesTest.php:238`, migration `2026_10_03_000003_nommer_les_generations_manquantes.php`.

D2 — Le catalogue mélange finition, version commerciale et variante technique

Les lignes `LE`, `Limited`, `NX 300h`, `320d` ou `RC 300 AWD` ne décrivent pas toutes le même axe. Certaines désignent l’équipement ; d’autres le moteur, une version commerciale ou la motricité.

Les tableaux du seeder contiennent puissance, cylindrée, cylindres et boîte, mais l’insertion des trims ne stocke pas ces colonnes. Certaines descriptions les reprennent en texte ; ce n’est pas une donnée structurée exploitable pour filtrer et valider.

Il ne suffit pas de déplacer ces nombres dans la table des trims : une finition peut offrir plusieurs groupes motopropulseurs. Le modèle `Motorisation` existant représente surtout des cotes de consommation officielles avec source et cycle ; il ne constitue pas encore un référentiel complet des configurations constructeur.

Évolution proposée : séparer conceptuellement modèle commercial, génération, variante technique et finition. Définir leurs relations et les marchés avant de décider de nouvelles tables. Une finition doit pouvoir correspondre à plusieurs variantes techniques.

Sources : `database/seeders/TrimsSeeder.php:34`, `:1463`, `app/Models/Trim.php:14`, `app/Models/Motorisation.php:8`.

D3 — Marchés, millésimes et périodes doivent être explicités

Le référentiel mélange des nomenclatures de plusieurs marchés sans rattachement structuré des finitions au marché ni période de disponibilité. Le pays d’importation ne suffit pas à identifier le marché pour lequel le véhicule a été configuré.

Les règles d’écriture contrôlent une année minimale et maximale, mais pas sa cohérence avec la génération choisie. Il faut toutefois distinguer fabrication, millésime et première immatriculation : appliquer un refus rigide sur des bornes approximatives pourrait exclure des véhicules réels.

Les 142 périodes ouvertes ne prouvent pas un retard : les tests préservent plusieurs productions hors Europe. **Aucune fermeture globale de périodes n’est recommandée.** Utiliser d’abord des avertissements et des dates assorties d’une source et d’un marché ; réserver le refus aux contradictions suffisamment établies.

Sources : `app/Http/Requests/StoreVehicleRequest.php:47`, `StoreOwnedVehicleRequest.php:59`, `tests/Feature/GenerationsNommeesTest.php:206`.

D4 — Les présélections techniques existent dans le schéma mais restent peu alimentées

Les colonnes `default_vehicle_type`, `default_seats`, `default_doors`, `default_transmission` et `default_power_hp` existent et sont exposées par la synchronisation. Le seeder des modèles ne les renseigne pas ; la création de modèle via `BrandController` ne les accepte pas non plus.

Les champs présents en base peuvent donc rester vides malgré la présence du modèle. Leur remplissage doit être prudent : une puissance « médiane » de génération ne doit pas devenir la caractéristique certaine d'un véhicule individuel. De même, les valeurs moteur/motricité par défaut d'une finition sont des aides à la saisie, pas une preuve de configuration.

Sources : migration `2026_09_01_100000_add_defaults_to_vehicle_models_table.php:15`, `VehicleModelsSeeder.php:500`, `app/Http/Controllers/Api/BrandController.php:68`, `app/Models/Trim.php:28`.

D5 — Une génération peut aussi regrouper plusieurs carrosseries

Le seeder donne une seule carrosserie par entrée modèle/génération. Par exemple, les entrées Mazda6 sont classées `berline` alors que le fichier explique le regroupement des berlines et breaks. Le partage de certains moteurs ne suffit pas à garantir des pièces de carrosserie communes.

Proposition : rendre explicite la carrosserie de la configuration ou du véhicule lorsque l'usage le nécessite. Une génération commune peut rester un regroupement utile sans signifier que toutes ses variantes ont les mêmes pièces. Le besoin est à croiser avec les filtres commerciaux et les familles de pièces réellement proposées.

Source : `database/seeders/VehicleModelsSeeder.php:351`.

6. Imports et résolution des identités

I1 — VIN et consommation ne choisissent pas toujours la même génération

Le VIN choisit la génération couvrant l'année dont le début est le plus récent. L'import des consommations prend le premier candidat couvrant l'année, sans ordre explicite de préférence. Les 70 transitions à année commune rendent cette divergence concrètement possible.

Le test intitulé « ne se chevauchent pas » autorise une fin égale au début suivant. Il ne résout donc pas l'ambiguïté des années inclusives. Aucun chevauchement strict au-delà de cette égalité n'a été trouvé dans le comptage des groupes marque–nom.

Proposition : partager une règle de résolution entre les chemins et conserver plusieurs candidats lorsque l'année seule ne permet pas de conclure. Une règle déterministe n'est pas une identification automobile certaine.

Sources : `app/Services/VinDecodeService.php:359`, `app/Console/Commands/ImportConsommations.php:869`, `tests/Feature/CatalogueModelesTest.php:50`.

I2 — L'import Europe peut laisser des générations et millésimes sans données

`modelesSansCote()` considère un nom comme couvert dès qu'une génération a une cote. Les autres générations de ce nom ne sont alors plus recherchées par ce mécanisme. La clé de couverture n'inclut pas la marque. De plus, la recherche Europe s'arrête au premier motif et au premier millésime contenant des données.

Conséquence : une commande qui trouve des données récentes n'a pas nécessairement complété les générations plus anciennes ni tous les alias.

Proposition : calculer les manques par marque, génération, année et variante, autoriser une période d'import explicite et produire un bilan des lignes non rattachées. L'impact exact sur les consommations actuellement stockées nécessite la base réelle.

Sources : `app/Console/Commands/ImportConsommations.php:389`, `:423`, `:755`.

I3 — NHTSA apporte des noms, pas un référentiel fin prêt à publier

L'import NHTSA crée des entrées au slug du seul nom, sans génération, période ni finition. Une entrée générique `corolla` peut coexister avec `corolla-e180` et `corolla-e210`. C'est un risque issu du mécanisme d'import, pas un doublon trouvé dans les seeders actuels.

La commande retourne aussi un succès global lorsque les appels HTTP non réussis sont seulement signalés puis ignorés. Le compteur ne distingue pas créations et mises à jour.

Proposition : importer dans une zone de rapprochement, produire un bilan explicite et valider les correspondances avant publication. Cet import ne doit pas être présenté comme une solution aux trous de finitions.

Source : `app/Console/Commands/ImportNhtsaCatalog.php:34`, `:54`, `:79`.

I4 — Les alias et les niveaux de confiance sont dispersés

L'import consommation possède des alias spécialisés qui ne sont pas partagés par le VIN. Celui-ci utilise notamment une égalité ou une sous-chaîne du nom ; des modèles voisins peuvent donc concurrencer le nom exact lors du choix par année.

Le niveau global de confiance VIN ne représente pas toutes les ambiguïtés, notamment plusieurs générations possibles ou certains retours NHTSA partiels. Il faut conserver les avertissements existants, puis définir une confiance et une justification par champ.

Proposition : dictionnaire d'alias partagé, limité par marque, marché et période ; priorité au nom exact et à l'alias vérifié ; rapprochement approximatif présenté comme suggestion.

Sources : `app/Console/Commands/ImportConsommations.php:290`, `app/Services/VinDecodeService.php:316`, `:517`.

7. Maintenance du catalogue et contrats API

M1 — L'ajout est mieux outillé que la correction

L'API permet d'ajouter un modèle et une finition, mais ne fournit pas de routes dédiées pour les modifier, fusionner ou désactiver. La création de finition ne permet pas de définir les IDs moteur et motricité par défaut. Les contraintes d'unicité existent en base, mais ne sont pas toutes anticipées par une validation métier.

Le seeder des trims ajoute uniquement les lignes absentes. Modifier la description, le rang ou les valeurs par défaut d'une ligne déjà présente dans ce fichier ne corrige donc pas la base lors du rejeu.

À l'inverse, les seeders des marques et modèles réimposent plusieurs champs, dont `is_active = true`. Une correction ou une désactivation faite par un autre canal peut être écrasée au rejeu correspondant. Il faut définir qui possède chaque champ : référentiel versionné, administration ou donnée importée.

Sources : `routes/api.php:220`, `app/Http/Controllers/Api/BrandController.php:68`, `database/seeders/TrimsSeeder.php:1481`, `VehicleModelsSeeder.php:500`, `BrandsSeeder.php:60`.

M2 — Les mêmes références ont plusieurs réponses partielles

La synchronisation des trims omet `description`, alors que `TrimResource` l'expose. En sens inverse, les valeurs moteur/motricité par défaut sont présentes dans la synchronisation mais absentes de cette ressource. Les ressources de modèles n'exposent pas tous les champs de statut et de présélection de la synchronisation.

La réponse du garage expose les IDs marque et modèle mais seulement les libellés pour plusieurs autres références, dont la finition. Cela ne suffit pas à reconstruire directement tous les IDs d'un formulaire d'édition. Le client peut disposer d'un contournement local : **son comportement reste à vérifier dans Flutter**.

La liste des modèles d'une marque ne filtre pas `is_active`, et sa ressource n'expose pas ce statut. La sélection de nouvelles références et la lecture d'anciennes fiches doivent suivre un contrat explicite.

Sources : `CatalogSyncController.php:58`, `app/Http/Resources/TrimResource.php:11`, `VehicleModelResource.php:11`, `OwnedVehicleResource.php:16`, `BrandController.php:63`.

M3 — Certains rejeux remplacent les compatibilités manuelles

Le seeder des filtres supprime toutes les compatibilités des références qu'il gère avant de les recréer, sans distinguer un ajout manuel d'une ligne qu'il possède. Une correction manuelle sur ces références peut donc disparaître lors de son rejeu. Ce seeder n'est pas assimilable à un rejeu quotidien de tous les référentiels : le risque concerne ses exécutions effectives.

Proposition : identifier la source propriétaire des lignes et ne remplacer que son périmètre. Le mécanisme `BackfillFitments` protège déjà les lignes déclarées ; ce comportement constitue un point de comparaison utile.

Sources : `database/seeders/FiltresHuileBoutiqueSeeder.php:671`, `app/Console/Commands/BackfillFitments.php:42`.

M4 — Le démarrage ne rejoue pas tous les prérequis du référentiel

La chaîne réelle du Dockerfile relance les modèles, les finitions et les points de contrôle, mais pas les marques, types moteur ou motricités. Si une nouvelle marque n'est ajoutée que dans les seeders du dépôt, le démarrage ne garantit pas sa création. Les modèles dont la marque n'existe pas sont ignorés ; les finitions dont le modèle n'est pas résolu le sont aussi.

À cette révision, les références statiques sont résolubles lorsqu'on prépare le catalogue complet. Le problème est donc un risque de déploiement partiel et un manque de bilan, pas une marque actuellement prouvée absente en production.

Proposition : définir une commande d'actualisation du référentiel dans l'ordre des dépendances et produire un résultat observable : nombres ajoutés, corrigés et ignorés, avec raisons. Le jeu `ReferenceDataSeeder`, qui crée des données génériques de test, n'est pas le référentiel principal et ne doit pas être ajouté à cette chaîne.

Sources : `Dockerfile:319`, `database/seeders/VehicleModelsSeeder.php:492`, `database/seeders/TrimsSeeder.php:1476`, `database/seeders/DatabaseSeeder.php:34`.

8. Ce qui fonctionne déjà et doit être conservé

- Les modèles et trims des seeders ne présentent pas de doublons de clés, et toutes les finitions pointent vers un modèle du même référentiel statique.
- Les couples marque–modèle et modèle–finition sont vérifiés à la création lorsque les champs concernés sont fournis.

- La saisie libre permet d’enregistrer un modèle absent ; les chaînes vides sont traitées et le garage ne doit pas perdre simultanément son modèle normalisé et son modèle libre.
- Une motorisation explicitement choisie pour un autre modèle est refusée, y compris dans plusieurs cas de mise à jour partielle.
- Un VIN hors période produit un avertissement ; une cote hors de toutes les périodes connues peut rester non rattachée.
- Des migrations et tests préservent les IDs lors du nommage de générations et évitent des doublons au rejeu.
- `BackfillFitments --fresh` protège les compatibilités déclarées.

Ces protections ne couvrent pas tous les défauts décrits. Les tests ont été lus, pas exécutés ; leur existence ne constitue pas un résultat de validation de cette instance.

9. Plan proposé, sans application

Lot	Travail	Résultat attendu	Critère de vérification	Ampleur indicative
A — Fiabilité	Timestamps et rattrapage sync ; validation de l’état final ; contrat/normalisation du code moteur	Les références transmises et enregistrées restent cohérentes	Reproductions avant/après sur sync incrémentale et modifications partielles ; vérification des consommateurs API	Moyenne
B — Mesure réelle	Audit en lecture seule du parc, garages, références et fitments	Liste des manques et anomalies pondérée par les véhicules concernés	Rapport chiffré avec IDs techniques, catégories d’écarts et aucune correction automatique	Faible à moyenne, selon l’accès
C — Compatibilités	États confirmé/à vérifier/inconnu ; traitement des sources synthétiques ; protection des enrichissements	Une donnée incomplète ne devient pas une certitude commerciale	Cas de moteurs différents d’une même génération et contrôle des sources	Moyenne à élevée
D — Enrichissement ciblé	Compléter les générations et finitions demandées, avec marché, années, source et statut	Moins de saisies libres et de choix approximatifs sur les véhicules utilisés	Lot relu, traçable, sans doublons, disponible en sync et rattaché sans perte d’IDs	Variable selon les sources
E — Outils et imports	Correction des références, file de validation des saisies libres, alias communs, imports par période	Maintenir et compléter le catalogue sans dépendre de correctifs ponctuels	Bilan d’import explicite, corrections préservées, scénarios d’édition vérifiés	Moyenne à élevée
F — Structure	Décider de la séparation physique modèle/génération/variante/finition	Schéma uniforme si les besoins le justifient	Plan de migration, correspondances d’IDs, réversibilité et contrat mobile validés	Élevée

Les ampleurs sont relatives, pas des estimations en jours. **Je recommande A et B en premier**, puis C et les premiers lots D selon les véhicules réellement utilisés. Le lot F nécessite une décision de conception distincte ; il n’est pas un préalable aux premiers correctifs.

Audit à effectuer sur la base réelle

1. Compter les véhicules et garages par marque, modèle, génération, année, finition et niveau de précision moteur.

2. Regrouper les valeurs `model_libre` par marque et fréquence, puis rechercher leurs correspondances sans écrire.
3. Rechercher les liens incohérents : modèle d'une autre marque, finition ou cote d'un autre modèle, références disparues ou inactives.
4. Mesurer les timestamps nuls ou anciens, puis comparer une synchronisation complète et incrémentale sur un jeu témoin.
5. Inventorier les codes moteur : caractère VIN, code complet, différences de casse, valeurs non reconnues.
6. Compter les cotes par génération et millésime, les lignes non rattachées et les années de transition ambiguës.
7. Mesurer la présence de fitments `generated` et les résultats dépendant d'un critère absent.
8. Vérifier dans Flutter les champs recopiés depuis le VIN, les IDs utilisés en édition et la gestion du cache.

Les exports devraient se limiter aux identifiants techniques et aux données nécessaires au diagnostic ; noms de personnes, contacts, VIN complets et plaques ne sont pas nécessaires au rapport de couverture.

Indicateurs utiles après correction

- Part des véhicules utilisés rattachés à une génération vérifiée, et part encore en saisie libre.
- Couverture des finitions vérifiées sur les générations réellement utilisées, par marché et période.
- Nombre de fiches présentant des liens incohérents.
- Nombre de changements de référence absents de la synchronisation incrémentale.
- Part des demandes de pièces confirmées, candidates ou indéterminées et critères manquants.
- Couverture des cotes par génération et année ; volume non rattaché.

L'objectif n'est pas de remplir artificiellement toutes les cases. Une valeur inconnue signalée est préférable à une génération, une finition ou une compatibilité erronée.

10. Annexes et reproduction

- `couverture-par-marque.csv` : 25 marques et leurs compteurs.
- `modeles-et-finitions.csv` : 248 entrées et les finitions associées.
- `donnees-catalogue.json` : données détaillées, lignes sources et transitions d'années.
- `compter_catalogue.py` : script du comptage statique.

Reproduction des annexes, sans accès à la base et sans modification du dépôt :

```
python /workspace/scratch/audit-classification-2026-10-21/compter_catalogue.py \
  --repo /workspace/autoparc-backend \
  --output /workspace/scratch/audit-classification-2026-10-21
```

Le script est adapté aux tableaux littéraux de cette révision. Sa fonction de slug couvre les caractères effectivement présents et ne prétend pas remplacer l'ensemble de la translittération Laravel. Si la structure des seeders change, le comptage doit être revérifié.

Les annexes décrivent les fichiers versionnés à ce commit ; elles ne certifient pas l'état de la base de production ou des caches mobiles.